

PA5 - Reflective Report & Member Contributions

Project: LoveYou – AI-Enhanced Dating Web Application

Course: Software Engineering (CS300 / CSC13002 - SE24C11 - Group 01)

Sprint: Sprint 5 (PA5-2026)

Submission Date: 23/08/2026

1. Executive Summary

Performed by: Nghĩa | ****Reviewed by:**** Chiến | **Edited by:** Nghĩa

Sprint 5 (PA5) represents the culmination of the LoveYou dating platform development cycle, transitioning the project from rapid feature construction into comprehensive quality assurance, generative AI functional validation, rigorous defect cataloging, and release stabilization.

Throughout this sprint, Group 01 achieved the following milestones:

- **Comprehensive Quality Assurance:** Formulated and executed **92 test cases** spanning 6 primary use cases (UC1 through UC6) and system Non-Functional Requirements (NFRs), achieving **72 Passed (78.3%)** and **20 Failed (21.7%)** outcomes.
 - **Systematic Defect Cataloging:** Discovered and classified **25 defects (BUG-01 to BUG-25)** across High, Medium, and Low severities in [BugReport.md](#), maintaining complete traceability with failed test cases.
 - **Generative AI Functional Validation:** Implemented schema-conformance, value-boundary, metamorphic, and semantic directional test strategies to rigorously evaluate Google Gemini AI features (Red-Flag safety analysis, question generation, and compatibility evaluation).
 - **Environment & Build Stabilization:** Resolved critical environment blockers (installing missing [@payos/node](#) dependencies and regenerating out-of-sync Prisma Client models) to deliver a robust **Build 3** final release package.
 - **Demonstration & Release Delivery:** Prepared an end-to-end user journey demonstration showcasing key workflows, system architecture, and Spec Kit integration.
-

2. Team Experience: Successes & Challenges

Performed by: Nghĩa, Chiến | **Reviewed by:** Hoàng, Văn | **Edited by:** Nghĩa

2.1 What Went Well

1. Exceeding Quality Assurance Scope:

The course rubric required a minimum of 5 use cases with at least 10 test cases each (totaling 50 test cases). The team exceeded this baseline by designing and executing 92 detailed test cases across 6 full use cases plus system NFRs, ensuring comprehensive coverage of authentication, discovery matching, realtime WebSocket chat, AI mini-games, administrative moderation, and citizen identity verification.

2. Methodological Rigor in Testing Generative AI:

Traditional deterministic assertions fail when validating generative AI outputs. The team developed a four-tier assertion methodology (Schema & Domain, Directional Correctness, Determinism of Guarded

Paths, and Graceful Degradation). By asserting semantic properties against ground-truth controlled scenarios (such as investment scams and coercive control dialogues), we verified that AI outputs met functional safety requirements without asserting fragile exact strings.

3. Engineering Integrity in Defect Logging:

Rather than modifying application code during the test cycle to force artificial 100% pass rates, the team prioritized transparency. All 20 failed cases and 25 defects were cataloged with exact reproduction steps, expected vs. actual outcomes, and explicit architectural rationales (such as deferring dual-token rotation and lockout mechanisms).

4. Agile Team Resilience and Dynamic Reallocation:

When member Nguyễn Tường Huy exhibited persistent unexcused absences and submitted inadequate work, the active team members swiftly redistributed responsibilities. Công Chiến absorbed AI testing methodology and backend execution; Trọng Văn took over chat and matching test cases; Minh Hoàng and Hoàng Tấn managed UI validation and video recording; and Trung Nghĩa coordinated tracking and documentation. This proactive reallocation prevented project delivery bottlenecks.

2.2 Challenges Faced & Root Causes

1. Cloud Database Network Latency on NFR Benchmarks:

The application utilizes a cloud-hosted Neon PostgreSQL database. Running API benchmarks from local development machines over the public internet introduced inevitable network round-trip latency (e.g., matching response averaged 4,882 ms against the <1 s benchmark). The root cause was testing against remote infrastructure rather than a containerized local database instance.

2. Contract Discrepancies and Specification Drift:

Certain backend implementations drifted from early Spec Kit contracts. For example, username login was supported in the service layer but rejected by the input validation schema ([BUG-05](#)), and the forgot-password endpoint returned HTTP 404 for unregistered emails instead of a non-enumerating HTTP 200 ([BUG-01](#)). Identifying these gaps required intensive cross-referencing between code, route handlers, and markdown contracts.

3. Generative AI Non-Determinism and Fallback Ambiguity:

Calibrating prompt outputs to produce distinct score separation between high-compatibility and low-compatibility game answers required iterative tuning ([BUG-10](#)). Additionally, when Gemini API keys were invalid or exhausted, the backend returned a polite mock fallback rather than explicitly flagging a degraded mode to the client ([BUG-21](#)).

4. Uneven Workload Due to Member Disengagement:

Compensating for a non-contributing teammate during the final sprint demanded significant overtime from the remaining five members, especially during test execution, defect verification, and multi-document synthesis.

3. Spec Kit Experience: Specification-Driven Development

****Performed by:****Chiến, Vũ Lê Trọng Văn | **Reviewed by:** Nghĩa | ****Edited by:****Chiến

Throughout PA3, PA4, and PA5, the team adopted **Spec Kit** as the core framework for Specification-Driven Development (SDD), structuring 12 distinct feature modules ([001-auth-authorization](#) through [012-live-support-chat](#)).

3.1 Benefits of Spec Kit Compared to Traditional Development

1. Clarity of Contract Before Implementation:

In traditional ad-hoc development, frontend and backend developers frequently negotiate API formats through informal chat messages, resulting in integration mismatches. Spec Kit enforced a strict contract-first workflow where data models, API endpoints, error responses, and user acceptance criteria were defined in markdown contracts before code was written. This significantly accelerated frontend mock integration.

2. Clear Task Decomposition and Traceability:

The automated decomposition from user stories into granular technical tasks (`tasks.md`) gave team members unambiguous implementation roadmaps. Dependencies between database migrations, backend controllers, and React views were visible before sprint execution commenced.

3. Structured Living Documentation:

Maintaining specs alongside the codebase in `src/specs/` prevented the common issue of documentation becoming obsolete. New team members or reviewers could trace any feature back to its original design intent and architectural constraints.

3.2 Limitations & Challenges Encountered

1. Superficiality of Auto-Generated Test Cases:

While Spec Kit generated initial test cases, auditing them revealed that they could not be trusted blindly. Many generated test cases were optimistic happy-path tests that assumed the code was correct rather than testing against the contract. For instance, the generated test for email validation accepted invalid strings, and negative edge cases (such as authorization boundaries, rate limiting, and state-transition violations) were largely omitted. Extensive human refinement was necessary to construct the 92 robust test cases in `TestCases.md`.

2. Specification Drift During Rapid Iteration:

When edge cases arose during implementation (e.g., handling rapid card clicks or refining OTP rate-limiting intervals), developers occasionally updated code directly without back-porting changes to the markdown specifications. This resulted in divergence between contracts and actual API behavior that had to be reconciled during PA5 testing.

3. LLM Context Limits on Complex Architectural Dependencies:

Spec Kit's AI generation struggled to account for cross-module side effects. For example, changing the user model to support Citizen ID verification status required migrations that impacted the matching algorithm and admin views, which the isolated module generator could not fully harmonize without human architectural intervention.

4. AI Coding Tools Usage: Effectiveness & Limitations

Performed by: Nguyễn Minh Hoàng, Lê Hoàng Tấn | ****Reviewed by:**** Chiến | **Edited by:** Nguyễn Minh Hoàng

Throughout the project lifecycle, the team actively utilized AI coding assistants—primarily **GitHub Copilot**, **ChatGPT**, and **Claude**—across architecture, frontend, backend, and quality assurance workflows.

4.1 Effective Aspects & Productivity Gains

1. Boilerplate & CRUD Code Acceleration:

AI assistants drastically reduced the time required to write standard Express route handlers, Prisma

CRUD queries, and repetitive React form handlers. Generating boilerplate for the 12 Spec Kit modules was estimated to be 40% faster than manual coding.

2. **Complex Regex & Parsing Generation:**

Authoring robust validation rules (e.g., regex patterns for Vietnamese Citizen Identity Cards, date formatting, email compliance) was completed rapidly with AI prompt assistance, needing only edge-case validation.

3. **Mermaid Diagram Drafting:**

ChatGPT was highly effective at translating architectural narratives and component structures into valid Mermaid syntax for C4 diagrams (System Context, Container, and Component levels) and database Entity-Relationship Diagrams (ERDs).

4. **Brainstorming Adversarial Test Inputs:**

Generating realistic ground-truth conversational datasets to test Gemini's red-flag safety analysis—specifically simulating coercive control dialogues and multi-stage financial phishing attempts—was greatly accelerated using AI prompts, providing realistic adversarial data for testing.

4.2 Limitations & Pitfalls Encountered

1. **Subtle Logical and Semantic Hallucinations:**

AI code generators frequently hallucinated nonexistent library methods or produced outdated API patterns. For example, Copilot occasionally suggested deprecated Express 4 middleware idioms or Prisma syntax incompatible with Prisma 7, requiring careful human debugging.

2. **Inability to Reason About Distributed State:**

When implementing the WebSocket-based realtime chat and in-memory mini-game sessions, AI tools consistently failed to address race conditions, connection reconnections, and state synchronization across multiple socket instances. Human architectural design was essential to ensure reliability.

3. **Over-Optimistic Code Assumptions:**

AI assistants exhibited a strong bias toward optimistic execution paths, routinely neglecting defensive programming checks (such as verifying whether an unmatched or block had occurred before emitting a socket message).

4. **False Sense of Test Coverage:**

When prompted to write tests, AI models tended to write circular tests that asserted existing implementation quirks rather than enforcing business specifications, reinforcing the principle that quality assurance requires critical human oversight.

5. SDLC Process Feedback & Constructive Recommendations

Performed by: Nghĩa,Chiến | **Reviewed by:** Whole Team | **Edited by:** Nghĩa

Based on our end-to-end experience across PA1 through PA5, the team offers the following constructive feedback on the course SDLC structure, tooling, and workflow:

5.1 PA Structure & Timeline Pacing

- **Staggering Specification vs. Implementation:**

In earlier phases, teams were tasked with authoring architecture, specifications, and substantial functional groups almost simultaneously. Staggering the milestones—requiring a formal contract freeze

milestone before sprint implementation begins—would allow teams to internalize Specification-Driven Development more deeply and avoid mid-sprint rework.

- **Early Performance & Benchmarking Guidelines:**
Performance NFRs (<1 s matching, <300 ms response times) were evaluated in PA5, but many teams adopted cloud-hosted databases (e.g., Neon, Supabase) that suffer from internet round-trip latency. Providing a standardized Docker Compose template for a local benchmarking database in PA2 would ensure teams measure raw application performance rather than public internet latency.

5.2 Tooling & Automated CI/CD Integration

- **Automated Specification Consistency Checkers:**
Introducing a lightweight linter or CI action to check whether routes and request payloads match Spec Kit contract markdown files would help prevent specification drift before the final testing phase.
- **Mandatory CI Pipeline from Sprint 3:**
Introducing an automated GitHub Actions pipeline in PA3 to run `prisma generate`, unit tests, and linting on every pull request would catch environment discrepancies (such as the uninstalled `@payos/node` dependency in [BUG-13](#) or stale Prisma Client in [BUG-14](#)) long before final packaging.

5.3 Team Governance & Accountability Checkpoints

- **Mid-Semester Formal Peer Accountability Review:**
The current grading structure evaluates individual contributions primarily at final submission (PA5). Introducing an official, graded mid-point peer evaluation (around PA3) with course staff intervention would provide disincentivized members with an early warning and give teams administrative support to resolve workload imbalances before the final deadline.

6. Individual Contributions & Personal Reflections

Performed by: All Team Members | **Reviewed by:** Nghĩa | **Edited by:** Nghĩa

6.1 Team Attendance & Workload Distribution

In accordance with team governance rules established in PA1 (Section 5: Accountability and Performance), individual contributions are evaluated based on actual task delivery, meeting attendance, and peer assessment.

Member	Student ID	Scrum 1 (14/08)	Scrum 2 (20/08)	Retrospective (23/08)	Attendance	Contribution
Nghĩa	19127484	Present	Present	Present	100%	100%
Nguyễn Công Chiến	23127331	Present	Present	Present	100%	100%
Vũ Lê Trọng Văn	20127095	Present	Present	Present	100%	100%

Member	Student ID	Scrum 1 (14/08)	Scrum 2 (20/08)	Retrospective (23/08)	Attendance	Contribution
Nguyễn Minh Hoàng	23127368	Present	Present	Present	100%	100%
Lê Hoàng Tấn	23127471	Present	Present	Present	100%	100%
Nguyễn Tường Huy	23127197	Absent	Absent	Absent	0%	30%

Note regarding Nguyễn Tường Huy: Member Huy was absent from all team Scrum meetings and submitted superficial, incomplete initial drafts for assigned AI testing tasks. All testing, defect logging, build stabilization, and documentation responsibilities were reassigned to and completed by active team members. His contribution score is penalized in accordance with PA1 governance rules.

6.2 Individual Reflections on Personal Contributions & Learning

1. Nghĩa (Student ID: 19127484 — Project Manager & Integration)

Throughout this capstone project, I served as Project Manager, overseeing agile sprint planning, backlog synchronization in Jira, Git repository integrity, and final document integration across all five project assignments. My key technical focus in Sprint 5 was synthesizing cross-functional artifacts, organizing the testing review, and ensuring strict compliance with course deliverables. Through this experience, I learned that effective project leadership is less about passive tracking and more about rapid adaptability—specifically in recognizing team member bottlenecks early, redistributing critical-path deliverables without rancor, and maintaining team morale under tight deadlines. Balancing engineering rigor with agile project management has been the most transformative takeaway of my academic journey.

2.Chiến (Student ID: 23127331 — Backend Lead & Test Lead)

As Backend and Test Lead, I designed the core Express server architecture, Prisma ORM schema models, authentication workflows, and the master test harness for PA5. In this final sprint, I authored the Master Test Plan, designed test suites for UC1, UC4, UC5, and NFRs, absorbed the AI evaluation testing methodology, and executed all 92 test cases while resolving critical runtime blockers. This project taught me the immense value of contract-first engineering over intuitive coding; discovering that our implementation subtly violated our own OTP and auth specifications proved that tests must be written against the specification rather than the code. Furthermore, formulating property-based assertions to validate non-deterministic Gemini LLM outputs expanded my perspective on modern AI quality assurance.

3. Vũ Lê Trọng Văn (Student ID: 23127476 — QA & Test Specialist)

My primary role in Group 01 was quality assurance and verification. In Sprint 5, I authored the test cases for Smart Matching (UC2) and Realtime Chat (UC3), verified reproduction steps for all 25 cataloged bugs, and performed comprehensive cross-referencing between test cases, execution logs, and defect reports. Working on this project taught me that testing is not merely a post-development checklist, but an adversarial

investigative discipline that uncovers boundary violations, unhandled socket state transitions, and authorization leaks. I gained deep practical skills in Supertest API probing, Jest assertion design, and the discipline of documenting bugs with reproducible steps and clear severities.

4. Nguyễn Minh Hoàng (Student ID: 23127368 — Frontend Lead)

As Frontend Lead, I was responsible for developing the user interface using React 18 and Vite, managing WebSocket socket connections for chat, and implementing the interactive matching card UI. In Sprint 5, I focused on UI polish, validating Gemini API fallback states, verifying administrative moderation panels, and directing the final video demonstration walkthrough. Through this project, I learned how to architect modular, maintainable React components and handle asynchronous states gracefully when backend and AI services experience degradation. Overcoming real-world integration challenges—such as debouncing rapid swipe gestures and synchronizing socket events—deepened my understanding of enterprise frontend development.

5. Lê Hoàng Tấn (Student ID: 23127471 — UI/UX & Frontend Developer)

My contributions centered on UI/UX design, building accessible frontend views, and managing the workflow for Citizen Identity Card verification (UC6) and Ice-breaking mini-games. During Sprint 5, I prepared visual test fixtures, evaluated responsive UI edge cases, conducted smoke tests on the Build 3 release, and contributed to the live demonstration script. This project provided me with invaluable experience in bridging user-centric design with complex technical constraints, such as client-side OCR error handling and interactive game canvas rendering. I learned that intuitive user experience depends heavily on how thoughtfully the frontend handles error boundaries and unexpected API states.

6. Nguyễn Tường Huy (Student ID: 23127197 — Architecture & AI Integration)

During the early phases of the project (PA1 to PA4), I contributed to initial use-case modeling, drafting C4 architecture diagrams, and exploring prompt designs for Gemini AI integration. However, during Sprint 5, due to personal circumstances and time management difficulties, I failed to maintain communication, missed all scheduled team meetings, and did not deliver the testing tasks assigned to me. Experiencing the consequences of my disengagement taught me a painful but essential lesson about individual accountability, professional ethics, and the dependency of team success on consistent participation. I recognize the unfair burden my inactivity placed on my teammates and have learned that proactive communication is mandatory in software engineering teams.

7. Key Lessons Learned & Future Roadmap

Performed by: Whole Team | **Reviewed by:** Nghĩa | **Edited by:** Nghĩa

1. Property-Based Testing is Mandatory for Probabilistic AI:

Standard unit testing paradigms fail when evaluating LLM responses. Testing must evaluate schema compliance, value range validity, and directional semantics (e.g., ensuring scam inputs score lower than polite conversations) to ensure functional reliability.

2. Defect Transparency Demonstrates Engineering Maturity:

Documenting 20 test failures and 25 defects honestly provides far greater engineering value and integrity than masking issues to produce artificial perfection.

3. Future Maintenance Priorities:

- Transition backend authentication to dual-token rotation (1-hour access + 7-day refresh JWT) (BUG-06).
- Implement rate-limiting brute force account lockout with 15-minute cooldown (BUG-08).
- Implement native touch/swipe gesture handling on mobile web clients (BUG-23).
- Containerize local database testing via Docker to isolate NFR latency benchmarks from public cloud network latency (BUG-15).